Lenguajes - Python

Cursada 2025

Archivos. Formatos JSON y CSV

Pensemos en las siguientes situaciones

¿Qué estructura usamos si queremos:

- guardar los puntajes cada vez que jugamos a un juego determinado?,
- tener un banco de preguntas para que cada vez que realizamos el repaso de clase pueda acotar por temas?,
- manipular los Python plus de los estudiantes por turnos?

¿Qué tienen todas estas situaciones en común?

Necesitamos una estructura que permita que los datos puedan **persistir** cuando la ejecución del programa finalice.

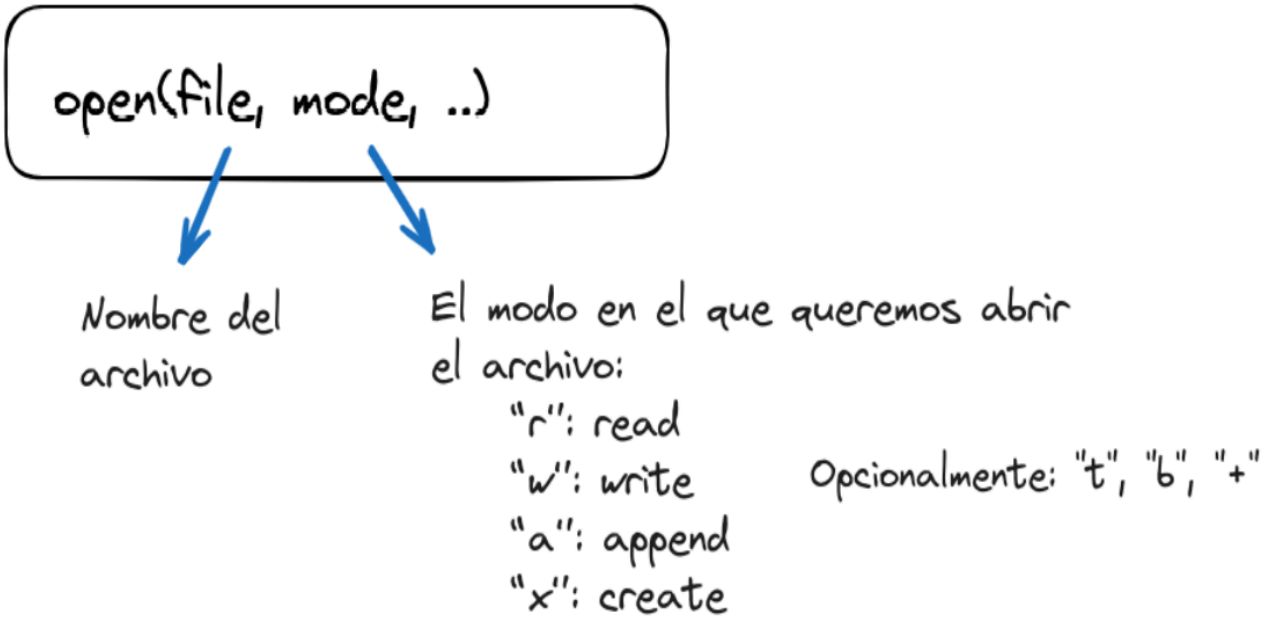
Algunas consideraciones antes de empezar

- Lo básico: ¿qué es un **archivo**?
- ¿Cómo podemos manipular los archivos desde un programa Python?

Manejo de archivos

- Existen funciones predefinidas.
- Si las operaciones fallan, se levanta una **excepción**.
- Los archivos se manejan como objetos que se crean usando la función open.

La función open



Veamos algunos ejemplos

Caso 1: usamos modo "w"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file1 = open('archivo.txt', 'w')
```

Caso 2: usamos modo "x"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file2 = open('archivo.txt', 'x')
```

Caso 3: usamos modo "a"

Veremos este caso más adelante...

Caso 4: ¿qué modo estamos usando?

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
file3 = open('archivo.txt')
```

En realidad la función open tiene más argumentos:

```
open(file, mode='r', buffering=-1, encoding=None,
     errors=None, newline=None, closefd=True, opener=None)
```

- **encoding**: sólo para modo texto. Por defecto, la codificación establecida en las configuraciones del sistema

```
notes = open("pp.xxx", "r+", encoding="UTF-8")
```

- En este caso, ¿debería llamarse pp.txt?
- ¿Cómo sé cuál es la configuración por defecto?

In [ ]:
```
import locale
locale.getlocale()
```

¿Qué pasa cuando el archivo está en otro directorio?

Usamos pathlib descripta en la PEP 428: una interfaz orientada a objetos que permite el manejo de rutas.

In [ ]:
```
from pathlib import Path
file_muchachos = Path('ejemplos') / "clase04" / "muchachos.txt"
file_muchachos
```

Archivos de texto vs. binarios

In [ ]:
```
the_best_song_file = open(file_muchachos,"rb")
lyrics = the_best_song_file.read()
the_best_song_file.close()
```

In [ ]:
```
print(lyrics)
```

In [ ]:
```
type(lyrics)
```

- El tipo **bytes** es una secuencia inmutable de bytes.
- Solo admiten caracteres ASCII.

In [ ]:
```
print(lyrics.decode('UTF-8')[:228])
```

- El tipo **str** de Python utiliza el estándar Unicode para representar caracteres.

- Generalmente la codificación predeterminada es **UTF-8**.
  - UTF significa **Unicode Transformation Format**, y el **8** significa que se utilizan valores de 8 bits en la codificación.

Una rosa azul ..

In [ ]:
```
file_rose = Path('ejemplos') / "clase04" / "rosa_azul.jpg"
rose = open(file_rose,"rb")
rose = rose.read()
```

In [ ]:
```
rose
```

¿Cómo almacenamos datos en un archivo?

El caso más sencillo: vamos a guardar solo texto plano.

In [ ]:
```
file_muchachos = Path('ejemplos') / "clase04" / "muchachos1.txt"
```

In [ ]:
```
f = open(file_muchachos, 'w')

cad1 = "De los pibes de Malvinas"
cad2 = "Que jamás olvidaré."

f.write("En Argentina nací")
f.write("Tierra del Diego y Lionel")
f.write(cad1)
print(f.write(cad2))

f.close()
```

- **write(cadena):** escribe *cadena* en el archivo y retorna cantidad de caracteres escritos.
- **close():** cierra el archivo.

Ahora si: abrimos con modo "a"

- ¿De **qué modo** se abre este archivo? ¿Qué significa?
- Luego de la instrucción, ¿**dónde se encuentra** archivo.txt?
- ¿De qué **tipo** es archivo.txt?
- ¿Cuándo puede dar un error esta sentencia?

In [ ]:
```
f = open(file_muchachos, "a")
f.write("****************************")
f.write("HOLA QUE TAL")
f.close()
```

¿Qué pasa si no existe?

In [ ]:
```
file_nuevo_muchachos = Path('ejemplos') / "clase04" / "muchachosNUEVO.txt"
```

In [ ]:
```
f = open(file_nuevo_muchachos, "a")
```

¿Cómo leemos los datos guardados?

In [ ]:
```
f = open(file_muchachos, 'r')
x = f.read(4)
print(f.read())
x
```

- **read(cantidad_bytes):** lee *cantidad_bytes* del archivo.
- Si cantidad_bytes es <0 o no está, lee hasta fin de archivo.
- Retorna "" si EOF.

CONSIGNA: probar en casa

Analizar el siguiente ejemplo que muestra otras formas de leer caracteres desde un archivo de texto.

In [ ]:
```
file_muchachos = Path('ejemplos') / "clase04" / "muchachos.txt"
```

In [ ]:
```
def by_characters():
    f = open(file_muchachos,"r")
    for x in f.read():
        print(x)
    f.close()

def by_lines():
    f = open(file_muchachos,"r")
    print(f.readlines())
    f.close()

def other_by_lines():
    f = open(file_muchachos,"r")
    for line in f:
        print(line)
    f.close()
```

In [ ]:
```
def main():
    print('Leo caracteres')
    by_characters()
    print('-' * 20)
    print('Leo lineas')
    by_lines()
    print('-' * 20)
    print('Otra forma')
    other_by_lines()
```

```python
if __name__ == "__main__":
    other_by_lines()
```

En resumen: - modos de apertura

- **'r'**: modo de lectura (**read**). Abre un archivo para lectura. **El modo por defecto, por lo que puede omitirse.**
  - Si el archivo **no existe**, se genera un error **FileNotFoundError**.
  - **Error**: intentar abrir un archivo que no existe o sin permisos.
- **'w'**: modo de escritura (**write)**. Abre un archivo para escritura.
  - Si el archivo **ya existe, se trunca** (se borra su contenido).
  - **Si no existe, se crea uno nuevo**.
  - **Error**: si no tengo permisos de escritura. Si no es el caso, no hay un error específico al abrir, pero se perderá el contenido del archivo si ya existía.

En resumen: - modos de apertura (cont.)

- **'a'**: modo para agregar (**append**). Abre un archivo para **agregar contenido al final**.
  - Si el archivo **no existe, se crea uno nuevo**.
  - **Error**: si el archivo **no tiene permisos de escritura**, se generará un error **PermissionError**.
- **'x'**: modo de **creación**. Abre un archivo para escritura, pero **solo si no existe**.
  - Si el archivo **ya existe**, se genera un error **FileExistsError**.
  - **Error**: intentar abrir un archivo que ya existe o si no tenemos permisos de escritura.

En resumen: - modos de apertura (cont.)

- **'r+'**: modo de lectura y escritura (read and write). Abre un archivo para lectura y escritura. **El archivo debe existir**; de lo contrario, se genera un error **FileNotFoundError**.
- **'w+'**: modo de lectura y escritura (write and read). Abre un archivo para lectura y escritura.
  - **Si el archivo ya existe**, se trunca.
  - **Si no existe**, se crea uno nuevo.
- **'a+'**: modo para agregar y lectura (append and read). Abre un archivo para agregar **contenido al final** y también permite la lectura.
  - Si el archivo **no existe**, se crea uno nuevo.
  - Si el archivo **no tiene permisos de escritura**, se generará un error **PermissionError**
- **'x+**: modo de creación. Crea un archivo para lectura y escritura. El archivo no debe existir.
  - Si el archivo **ya existe**, se genera un error **FileExistsError**.

En resumen: tipos de archivos

- **'b'**: **binario** (binary). Se utiliza al especificar el modo de apertura (por ejemplo, 'rb', 'wb', 'ab') para abrir archivos en formato binario.
- **'t'**: **texto** (text). Se utiliza al especificar el modo de apertura (por ejemplo, 'rt', 'wt', 'at') para abrir archivos en formato texto. **Es el formato por defecto, por lo que puede omitirse.**

¿Qué pasa si necesito guardar información que tiene una estructura?

- Pensemos en estos ejemplos:
  - Los puntajes cada vez que juego a un juego. Información tipo: **nombre jugador, puntaje, fecha.**
  - El banco de preguntas:**tema, enunciado, respuesta correcta.**
  - Los Python plus de los estudiantes por turnos:**turno, nombre, apellido, num_legajo, cantidad_puntos, etc.**
- En estos casos también podría usar un **archivo de texto**. ¿Cómo se les ocurre?

Algunas posibilidades

```
'Tema: Photograph - Intérprete: Ed Sheeran - Año: 2015'
---
Tema: Photograph
Intérprete: Ed Sheeran
Año: 2015
---
'Photograph-Ed Sheeran-2015'
'Photograph**Ed Sheeran**2015'
```

- ¿Pros y contras?

Hay otras formas mejores...

JSON (JavaScript Object Notation)

- Es un **formato de intercambio de datos** muy popular. Por ejemplo:

```
{ "Tema": "Photograph",
  "Intérprete": "Ed Sheeran",
  "Año": 2015}
o
 [{"Tema": "Unintended",
  "Intérprete": "Muse",
  "Año": 1999},
  {"Tema": "Photograph",
  "Intérprete": "Ed Sheeran",
  "Año": 2015}]
```

- +Info

¿Sólo para archivos?

- Veamos este ejemplo: https://sampleapis.com/api-list/movies

Y éste es solo uno de los tantos ejemplos ...

Módulo json

- Python tiene un módulo que permite trabajar con este formato.
- Para usarlo, debemos importarlo.

In [ ]:
```
import json
```

- Permite serializar objetos.
  - serializamos con: **dumps()** y **dump()**.
  - deserializamos con: **loads()** y **load()**.
- Más info en: https://docs.python.org/3/library/json.html

DESAFIO 1

> Queremos guardar en un archivo información de la música que más me gusta.

Vamos a generar un archivo con la siguiente estructura:

- nombre de la banda o intérprete,
- ¿es_solista?
- ciudad y país en donde procede,
- una referencia a su trabajo.

Empecemos por los datos

¿Qué estructura de Python les parece mejor para almacenar los datos antes de querer guardarlos en un archivo?

In [ ]:
```
data = [
    {"nombre": "William Campbell", "es_solista": False, "ciudad": "La Plata", "pais":
    {"nombre": "Marcos Fava", "es_solista": True, "ciudad": "La Plata", "pais": "Arge
    {"nombre": "Ed Sheeran", "es_solista": True, "ciudad": "Halifax", "pais": "Reino
    {"nombre": "INXS", "es_solista": False, "ciudad": "Sidney", "pais": "Australia","
    {"nombre": "La Ley", "es_solista": False, "ciudad": "Santiago", "pais": "Chile",
    {"nombre": "Héroes del Silencio", "es_solista": False, "ciudad": "Zaragoza", "pai
    {"nombre": "Panza", "es_solista": False, "ciudad": "Buenos Aires", "pais": "Argen
    {"nombre": "Adele", "es_solista": True, "ciudad": "Tottenham", "pais": "Reino Uni
    ]
```

In [ ]:
```
type(data)
```

In [ ]:
```
file_route = Path('ejemplos') / "clase04" / "musica.json"
music_file = open(file_route, "w")
```

Ahora, vamos a guardar nuestros datos en el archivo:

Necesitamos **serializar** el objeto referenciado con **data**: usamos **json.dump**

In [ ]:
```
import json

json.dump(data, music_file)
music_file.close()
```

DESAFIO 2

> Queremos visualizar los datos sobre mi música que previamente guardamos.

Ahora necesitamos **deserializar** para trabajar con nuestra lista de diccionarios: usamos **json.load**

In [ ]:
```
#Recordemos que:
#file_route = Path('ejemplos') / "clase04" / "musica.json"

file = open(file_route, "r")
data = json.load(file)
print(data)
file.close()
```

¿De qué tipo de **data**?

In [ ]:
```
type(data)
```

Abramos el archivo y observemos el dato booleano: ¿algo raro?

- Representación en Python
  - En Python, los valores booleanos se representan con **True** y **False** (con la primera letra en mayúsculas)
- Representación en JSON
  - En JSON, los valors booleanos se representan con **true** y **false** (en minúsculas)

Un poco más legible

In [ ]:
```
pretty_data = json.dumps(data, indent=4)
#print(pretty_data)
```

¿De qué tipo es **pretty_data**?

In [ ]:
```
type(pretty_data)
```

En resumen - JSON

El **módulo json** permite acceder a las siguientes funciones:

- **json.dump():** serializa un objeto Python a un objeto en formato JSON que puede almacenarse en un archivo.
- **json.load():** deserializa un objeto JSON almacenado en un archivo a un objeto Python.

In [ ]:
```
file_json = open("tempo.json", "w")
data = [{'first_name': 'Hilda', 'last_name': 'Lizarazu'}, {'first_name': 'Fabiana', '
```

```
json.dump(data, file_json)
file_json.close()
```

In [ ]:
```
file_json = open("tempo.json")
data =  json.load(file_json)
data
```

En resumen - JSON (cont.)

El **módulo json** permite acceder a las siguientes funciones:

- **json.dumps():** serializa un objeto Python a un objeto **str**.
- **json.loads():** deserializa un objeto JSON (puede ser str o byte) a un objeto Python.

In [ ]:
```
data = [{'first_name': 'Hilda', 'last_name': 'Lizarazu'}, {'first_name': 'Fabiana', '
data_str = json.dumps(data)
data_str
```

In [ ]:
```
data =  json.loads(data_str)
data
```

Usa las siguientes tablas de conversión:

- json-to-Python
- Python-to-json

CSV: ¿más formatos?

- CSV (Comma Separated Values).
- Es un formato muy común para importar/exportar desde/hacia hojas de cálculo y bases de datos.
- Ejemplo:

```
nombre,es_solista,ciudad,pais,referencia
William Campbell,False,La
Plata,Argentina,www.instagram.com/williamcampbellok
Marcos Fava,True,La Plata,Argentina,https://open.spotify.com/intl-
es/artist/2risap4rX5qNM54Gdiiuf1
Ed Sheeran,True,Halifax,Reino Unido,https://open.spotify.com/intl-
es/artist/6eUKZXaKkcviH0Ku9w2n3V
```

- +Info - PEP 305

Datasets

Antes que nada: ¿qué es un dataset?

- Hay muchos datasets disponibles de muchas temáticas:
- Datos de gestión de gobiernos:
  - Har un portal de datos de Argentina, de CABA, de pcia. de Buenos Aires.
  - Datos del Banco mundial
  - Portal Kaggle
  - IMDB

    - Y muchos más...

- Muchos son datos abiertos, pero otros... no tanto...
- ¡PRESTAR ATENCIÓN a la licencias y requisitos para su uso!
- Muchos en formato **CSV** y otros en **JSON**

Nuestra música en Spotify

Vamos a trabajar con el archivo: songs_normalize.csv

- Tenemos una copia de la descripción del dataset en donde se explica a qué se refiere cada campo.

In [ ]:
```
import csv
file_route = Path('ejemplos') / "clase04" / "songs_normalize.csv"
```

In [ ]:
```
file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')
```

In [ ]:
```
#encabezado = csvreader.__next__()
header = next(csv_reader)
print(header)
```

In [ ]:
```
file.close()
```

El módulo csv

- Hay que importarlo.
- **csv.reader**: crea un objeto **iterador** que nos permite recorrer las líneas del archivo.
- ¿Por qué incluimos el paŕametro **delimiter**? ¿Dialectos?

```
csv_reader = csv.reader(file, delimiter=',')
```

DESAFIO 3

> Busquemos cuántos temas hay de Adele en el dataset de Spotify.

Comencemos nuevamente el proceso.

In [ ]:
```
file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')

#header = csvreader.__next__()
header = next(csv_reader)
print(header)
```

In [ ]:
```
for line in csv_reader:
    if line[0] == "Adele":
        print(f"{line[1]:<40} {line[17]}")
file.close()
```

- ¿Cómo se accede a cada dato?
- ¿De qué tipo son header y line?

In [ ]:
```
type(header)
```

Otra solución ...

In [ ]:
```
file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')

adele_songs = filter(lambda x:  x[0] == "Adele", csv_reader)
for elem in adele_songs:
        print(f"{elem[1]:<40} {elem[17]}")

file.close()
```

Otra forma de acceder: csv.DictReader

In [ ]:
```
file = open(file_route, "r")
csv_reader = csv.DictReader(file, delimiter=',')

adele_songs = filter(lambda x:  x["artist"] == "Adele", csv_reader)

for elem in adele_songs:
        print(f"{elem['song']:<40} {elem['genre']}")

file.close()
```

Hagamos la siguiente modificación a nuestro código:

> Agregar un título al listado: "Adele en Spotify".

In [ ]:
```
file = open(file_route, "r")
csv_reader = csv.DictReader(file, delimiter=',')
```

In [ ]:
```
adele_songs = filter(lambda x:  x["artist"] == "Adele", csv_reader)
print("***Adele en Spotify")
for elem in adele_songs:
        print(f"{elem['song']:<40} {elem['genre']}")
#file.close()
```

No nos gusta y queremos agregar una tabulación para que que quede mejor. Modifiquemos y volvamos a ejecutar...

¿Qué observamos?

Recordemos que ...

> csv_reader es un **iterador** por lo que cuando llegamos al final de la iteración ya no vuelve a comenzar.

En un ejemplo más adelante veremos otra forma de trabajar teniendo en cuenta esto.

Creamos un archivo csv con nuestra música

- **csv.writer:** retorna un objeto que convierte los datos con los que trabajamos en el programa en cadenas con el formato delimitadas con el separador correspondiente.

In [ ]:
```
file_route_json = Path('ejemplos') / "clase04" / "musica.json"
file_route_csv = Path('ejemplos') / "clase04" / "musica.csv"
```

In [ ]:
```
#import csv
#import json

file_json = open(file_route_json)
file_csv = open(file_route_csv, "w")
```

In [ ]:
```
music = json.load(file_json)
writer = csv.writer(file_csv)
```

¿Qué contiene **music**? ¿De qué tipo es?

In [ ]:
```
type(music)
```

In [ ]:
```
# Grabo el encabezado con los nombres de las columnas
writer.writerow(["nombre", "es_solista", "ciudad", "pais", "referencia"])
#Grabo los datos
for elem in music:
    writer.writerow( [elem["nombre"], elem["es_solista"], elem["ciudad"], elem["pais"

file_json.close()
file_csv.close()
```

Exploremos el archivo .... musica.csv

Ahora lo leemos desde Python

In [ ]:
```
#Recordemos que
#file_route_csv = Path('ejemplos') / "clase04" / "musica.csv""
file_cvs = open(file_route_csv, "r")
csv_reader = csv.reader(file_cvs, delimiter=',')
for line in csv_reader:
    print(line)
file_csv.close()
```

¿Notan algo respecto a los datos? Observemos el campo booleano.

En resumen - CSV

El **módulo csv** permite acceder a las siguientes funciones:

- **csv.reader(csvfile):** retorma un objeto iterable **reader** para procesar las líneas del archivo csvfile.
- **csv.writer(csvfile):** retorna un objeto **writer** responsable de convertir los datos del usuario en cadenas delimitadas en archivo csvfile.

In [ ]:
```
import csv
file_csv = open("tempo.csv", "w")

writer = csv.writer(file_csv)
writer.writerow(['first_name', 'last_name'])
writer.writerow(["Hilda", "Lizarazu"])
writer.writerow(["Fabiana", "Cantilo"])
file_csv.close()
```

In [ ]:
```
file_csv = open("tempo.csv")
csv_reader = csv.reader(file_csv, delimiter=',')
for line in csv_reader:
    print(line)
```

En resumen - CSV (cont.)

El **módulo csv** permite acceder a las siguientes funciones:

- **csv.DicReader(csvfile):** retorna un objeto **reader** pero asigna la información de cada fila a un diccionario cuyas claves se proporcionan mediante el parámetro opcional fieldnames o extraídos de la primera fila.
- **csv.DictWriter(csvfile):** crea un objeto **writer** pero utilizando un diccionario para generar las fileas en el archivos csvfile.

In [ ]:
```
file_csv = open("tempo.csv", "w")
fieldnames = ['first_name', 'last_name']
writer = csv.DictWriter(file_csv, fieldnames=fieldnames)
writer.writeheader()
writer.writerow({'first_name': 'Hilda', 'last_name': 'Lizarazu'})
writer.writerow({'first_name': 'Fabiana', 'last_name': 'Cantilo'})
file_csv.close()
```

In [ ]:
```
file_csv = open("tempo.csv")
reader = csv.DictReader(file_csv)
print(list(reader))
for row in reader:
    print(row['first_name'], row['last_name'])
```

JSON vs. CSV

- ¿Con qué tipo de datos trabajan? (¿texto o binario?)

- ¿En qué casos usamos archivos en formato json?

- ¿En qué casos usamos archivos en formato csv?

La sentencia with

La sentencia with crea un objeto denominado **runtime context** o **contexto de tiempo de ejecución** que permite ejecutar un grupo de sentencias bajo el control de un **administrador de contexto** o **context manager**. ¿Qué es esto?

> Un administrador de contexto permite asignar y liberar recursos cuando se desee.

El ejemplo típico se da en el acceso a archivos, ya que son **recursos externos** a nuestros programas que requieren una gestión adecuada.

In [ ]:
```
with open(file_route_csv)  as file_csv:
    csv_reader = csv.reader(file_csv, delimiter=',')
    header, data = next(csv_reader), list(csv_reader)

print(header)
for line in data:
    print(line)
```

- ¿Dónde se cierra el archivo?
- ¿Por qué les parece que trabajamos **data** como una lista?

CONSIGNA: tarea para el hogar

> Dado el conjunto de datos de Spotify, queremos:

> 1- Guardar en otro archivo, en formato json, las canciones que tienen asignado más de un género.

> 2- Los cinco (5) artistas con más canciones en el dataset durante el año 2019.

In [ ]:
```
# Solución
```

Exploremos el siguiente dataset de películas

In [ ]:
```
file_route = Path("ejemplos") / "clase04" / "mymoviedb.csv"

with open(file_route, "r") as file_csv:
    csv_reader = csv.reader(file_csv)
    header, data = next(csv_reader), list(csv_reader)
```

In [ ]:
```
header
```

In [ ]:
```
data[:1]
```

DESAFIO 4

> Queremos ver qué películas tienen más de 9980 votos.

Deberíamos trabajar con la columna Vote_Count (columna 4)

In [ ]:
```
movies = filter(lambda x: x[4] > "9980", data)
for movie in movies:
    print(movie[1])
```

- ¿Hay algún problema?

Probar en casa: convertir a int x[4]. **Spoiler:** da error. ¿Por qué?

DESAFIO 5

> Queremos ver los idiomas de las películas.

In [ ]:
```
#header
```

In [ ]:
```
languages = map(lambda x: x[6], data)
print(list(languages))
```

¿Nos interesa que se repita?

In [ ]:
```
languages = set(map(lambda x: x[6], data))
```

In [ ]:
```
print(languages)
```

Notamos que hay datos que no son correctos.

Más adelante veremos cómo hay otras formas de procesar estos datasets

DESAFIO 6

> Queremos descargar el poster de las Spiderman.

In [ ]:
```
data[:1]
```

In [ ]:
```
spiderman = data[:1][0]
poster_spiderman = spiderman[8]
poster_spiderman
```

In [ ]:
```
import requests
image_route = Path("ejemplos") / "clase04" / "poster_spiderman.jpg"

image = requests.get(poster_spiderman)
with open(image_route, 'wb') as f:
    f.write(image.content)
```

> El módulo request permite realizar peticiones HTTP.

En este caso, estamos realizando un GET de un recurso ubicado en la URL dada: 'https://image.tmdb.org/t/p/original/1g0dhYtq4irTY1GPXvft6k4YLjm.jpg'

CONSIGNA: tarea para el hogar

Dados los módulos de juegos mostrados en la clase, modificar el código para que:

1. Se puedan jugar cada juego en forma independiente o desde el módulo juegos.
2. Si se juega desde el módulo juegos, guardar en un archivo el día, hora en que se jugó cada juego.

Para la parte 2, deben elegir cuál es la mejor estructura y tipo de archivo utilizar.

- Si quieren, subir el código modificado a su repositorio en GitHub y compartir el enlace a la cuenta @clauBanchoff Y @sofiamartin.

Seguimos la próxima ...